Integrating Pivoted QR for Coreset Selection in Variational Continual Learning

Marcelina Marjankowska

Abstract

Addressing the challenge of catastrophic forgetting in continual learning tasks, this study introduces a QR factorization with column pivoting based coreset selection technique for enhancing Variational Continual Learning (VCL) [26]. Moreover, we explore the VCL approach in Convolutional Neural Networks. We conduct an empirical evaluation comparing the efficiency of QR against conventional coreset selection methods across a series of sequential learning tasks with the MNIST, notMNIST and CIFAR-10 datasets. Our findings reveal that QR leads to improvements on the harder datasets and performs well on its own.

1 Introduction

Continual learning (CL) [29], or lifelong learning, focuses on developing AI algorithms that learn from sequential data over time, adapting to new data distributions, class changes, and new tasks, mirroring the ever-evolving real-world scenarios where retraining is often infeasible and expensive. Challenges include *catastrophic forgetting* [24], where new learning can erase previous knowledge. Solutions involve regularization [19], architectural changes [31], and replay mechanisms [30] to balance new data adaptation (*plasticity*) and preservation of learned information (*stability*).

In this work we consider the Variational Continual Learning (VCL) approach proposed by Nguyen et.al. in [26]. They note, employing a Bayesian perspective, that the previous parameter posterior can be used as a prior after the new data arrival. Since calculating exact posterior is intractable, a projection onto simpler distribution is employed resulting in an approximation. Use of *coresets* - small samples of previous data - is proposed as an enhancement

Despite numerous coreset selection techniques available, their application in VCL, remains underexplored. Achieving optimal continual learning (CL) is an NP-hard task that demands perfect memory [20], which underscores the significant role of episodic memory quality in boosting model effectiveness. In this work, we consider the QR factorization with column pivoting (QR) as a procedure alternative to the popular K-Center algorithm. Additionally, we employ the VCL approach on a Convolutional Neural Network (CNN), to further investigate the coreset properties and potential of this variational approach. The experiments are performed in task-incremental settings on MNIST, notMNIST and CIFAR-10 datasets.

2 Related work

Variational Continual Learning was further developed in recent years. In [12] Farquhar and Gal introduce a Bayesian approach that doesn't rely on posterior from earlier tasks. Instead, it adapts the model by modifying the likelihood term. Further, Generalized VCL [23], enhances VCL by incorporating likelihood-tempering and task-specific FiLM layers, significantly improving performance and calibration compared to prior approaches. This method blends the strengths of VCL and Online EWC, effectively addressing the over pruning issue in variational inference.

Mini-project, A18724S1 Uncertainty in Deep Learning 2023/2024

Coreset selection methods are used to summarize datasets and reduce computational load, as outlined in recent survey [14]. Within the realm of CL, coresets offer a promising solution to balance between retaining important information from past tasks and limiting memory requirements [2, 15]. Other applications of coreset methods include data-effective learning [34, 25], active learning [35, 32], robust learning against noise [18, 25], clustering [2, 3] and efficient GAN training [33]. In Continual Learning, using coresets is analogous to *episodic memory*, leveraging selective memory retention techniques to mitigate forgetting, as discussed by Jung et al. in [17] and by Li and Hoiem in [22]. Rebuffi et al. [28] refine the approach with iCaRL, utilizing coreset selection for incremental learning.

QR Factorization with Column Pivoting algorithm (QR-CP) [6] is widely used in the numerical linear algebra community, admitting fast and reliable implementations [4, 36] and extensive literature [13, 10, 1]. It also finds applications in model order reductions (Q-DEIM procedure) [9, 27], least squares problems [6, 8], and feature selection [7], constantly inspiring new applications. QR factorization, utilized for dimensional reduction, shares similarities with PCA which, in turn, relies on the SVD decomposition [16]. However, unlike PCA's feature extraction, QR-CP's permutation matrix enables feature selection by identifying representative columns. In this work we consider an unusual approach of data point (and not feature) selection for dataset representation. As for my knowledge, QR-CP has not been applied in this way before.

3 Theoretical Background

The VCL framework introduced in [26] is general and can be adapted to continual learning settings such as task-incremental and class-incremental settings, generative tasks and non i.i.d data. Throughout this section, we follow their theoretical derivations. In current setting we aim to learn parameters of a Bayesian Neural Network (BNN) using a series of datasets $\mathcal{D}_t = \{(x_t^{(i)}, y_t^{(i)})\}_{i=1}^{N_t}$, with potentially different distributions. We concentrate here on multi-task learning, where each $\mathcal{D}_t$ corresponds to a task, within which data is i.i.d.

BNNs leverage the predictive distribution $p(\hat{y}|x, D) = \mathbb{E}_{p(\theta|D)}[p(\hat{y}|x, \theta)]$. This setup allows for sampling weights and biases from $p(\theta|D)$, leading to predictions $\hat{y}$. See Appendix A.1 for a walk-through example of approximating posterior with variational inference. In VCL, upon training on dataset $\mathcal{D}_t$, the posterior distribution from the previous task $p(\theta|\mathcal{D}_{1:t-1})$ acts as a prior. The property

$$p(\theta|\mathcal{D}_{1:T}) \propto p(\mathcal{D}_T|\theta)\, p(\theta|\mathcal{D}_{1:T-1}) \qquad \text{(See Appendix A.2for derivation.)}$$

enables posterior computation without accessing previous data $\mathcal{D}_{1:T-1}$. For computational feasibility, this posterior is approximated as a multivariate Gaussian. With some fixed $q_1(\theta)$, the approximate posteriors $q_t(\theta) \approx p(\theta|\mathcal{D}_{1:t})$ can be recursively defined by

$$q_t(\theta) = \text{proj}[q_{t-1}(\theta)p(D_t|\theta)]$$

where various projection operators are possible. Here, we use the *variational KL minimization* onto space of multivariate Gaussians. Thus, at each dataset's arrival we maximize

$$\mathcal{L}(q_t(\theta)) = \mathbb{E}_{q_t(\theta)}[\log p(\mathcal{D}_t|\theta)] - \text{KL}(q_t||q_{t-1}) \tag{1}$$

with respect to parameters defining q_t (see Appendix A.3 for derivation). In the above objective, KL-divergence can be calculated explicitly, while the derivative of the first term is calculated via MC integration and the local reparametrization trick.

To further mitigate the issue of catastrophic forgetting and accumulation of approximation errors, the authors of [26] propose keeping a small representative set of data from previous tasks, termed *coresets* $C_t \subset \mathcal{D}_{1:t}$. The training focuses on $\mathcal{D}_t \backslash \mathcal{C}_t$ and the posterior from non-coreset data

$$p(\theta|\mathcal{D}_{1:T}) \propto p(\theta|\mathcal{D}_{1:T}\backslash\mathcal{C}_t)\, p(C_t|\theta)) \tag{2}$$

$$p(\theta|\mathcal{D}_{1:t}\backslash C_t) = p(\theta|\mathcal{D}_{1:t-1}\backslash C_{t-1})\, p(\mathcal{D}_t \cup C_{t-1}\backslash C_t|\theta). \tag{3}$$

admitting two approximations using again the variational KL projection, with $\tilde{q}_t(\theta) \approx p(\theta|\mathcal{D}_{1:t}\backslash C_t)$ being optimized as in 1 and q_t used for predictions.

Any matrix $A \in \mathbb{R}^{n\times m}$ can be factorized as $AP = QR$ with $Q \in \mathbb{R}^{n\times n}$ an orthogonal matrix, $R \in \mathbb{R}^{n\times m}$ upper-triangular with non-increasing diagonal entries and $P \in \mathbb{R}^{m\times m}$ a permutation matrix that reorders columns of A [13, Alg. 5.4.1]. QR factorization is usually employed to construct

orthogonal spaces of the data, with its truncated version used for projections onto lower-dimensional spaces. Consider D, a data matrix, where columns represent features and rows denote data points. In the QRCP of D, matrix P identifies a maximally independent feature space. Conversely, applying QRCP to D^T suggests a basis consisting of data points, that would represent all of them well.

4 Proposed Approach

The goal of the study is to investigate the use of QR-CP algorithm for coreset selection in the VCL setting. In addition to generally, investigating impact of coreset method choice on performance. We will focus on networks with shared lower model architecture and separate task-specific head layers. We consider using a combination of Bayesian linear and convolutional layers. The layers are parametrized by weight mean and variance being updated using the loss 1. The prior over parameters is updated after each batch following the equations 2 and 3.

We start by replicating the setting and task-incremental experiments of the original VCL paper [26] employing a Neural Network on the MNIST and notMNIST datasets. We continue to consider a harder CIFAR-10 dataset of colour object images; again in a task-incremental setting of five binary classification tasks arriving sequentially. A Convolutional Bayesian Neural Network is implemented to account for the dataset complexity. We employ classical baselines in addition to VCL models enhanced with random, K-Center and QR coresets. We note the average model accuracy and at each time step evaluate the model on all the previous tasks. We finish with coreset visualisations to verify the intuition about base-building property of QR.

5 Experiments

We run the experiments in task-incremental settings using the MNIST, notMNIST and CIFAR-10 datasets [1]. The datasets are split into five binary classification tasks i.e. for notMNIST it would be A/B, C/D, E/F, G/H and I/J. We observe performance of the VCL, and VCL aided with coresets. The coreset methods considered are random selection, greedy K-Center [32] and QRCP (see Appendix B.1 for details). As a baseline, we also consider models trained only on coresets, but following the same architecture (Appendix B.2).

Multi-head Bayesian Neural Network with VCL updates was implemented from scratch, following the setting of [26]. The convolutional model consists of 3×3 Bayesian convolutional layers followed by 2×2 Max Pooling, a fully-connected Bayesian layer and task-specific head layers (see Appendix B.1). The parameter priors are updated for lower layer and task-specific ones, admitting the loss 1. BNN and BCNN weight priors are set to mean 0 and variance 0.01. All weights initial variance is set to 10^{-6} and mean is drawn from $\mathcal{N}(0, 0.1)$. Bias values are drawn from $\mathcal{N}(0, 0.1)$. Models are trained using the Adam optimizer with learning rate 10^{-3}.

Split (not)MNIST Experiments. The experiment Split MNIST is an incremental task of distinguishing between pairs of hand-written digits [37]. Following [26] we use a multi-head BNN with VCL updates, 2 hidden layers with 256 units each and ReLU activation. The training is done with batch sizes 256 and 100 epochs. For the coresets we choose 40 examples from each task. Methods that use only coresets run 100 epochs on a single batch and are trained using the VFE method as well. The notMNIST [5] is a more challenging dataset with 40,000 images of characters from A to J. A simple NN with 4 layers with ReLU activation and 150 hidden units is used. Parameters are chosen as above. Figures can be found in Appendix B. Experiments reflect the results of [26], with notable case of catastrophic forgetting in VCL. QR doesn't improve performance of corseted VCL, but on its own presents impressive performance producing an average accuracy of 0.9.

Split CIFAR-10. The CIFAR-10 [21] dataset contains images of objects in ten classes, each containing 6,000 coloured 32x32 images. In the Split CIFAR-10 task [28] the data is divided into five pairs of two classes, each creating a binary classification. Being a much harder dataset, naturally performance on NN was unsatisfactory, and so we deploy a convolutional architecture. The lower layer is convolutional (see Appendix B.1) and the task-specific heads consist of one layer of size 150. The parameters of both linear and convolutional layers consist of weight mean and variance

[1] All used code can be found in `https://github.com/sdfghjklsdfghjkldfghjkl/udl_miniproject`.

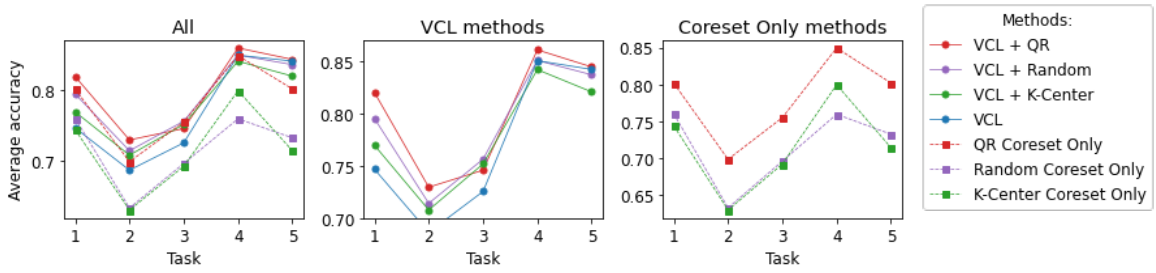


Figure 1: Split CIFAR-10 experiment with VCL based-methods and corset variations. Plots show average accuracy for each task over the five model iterations.

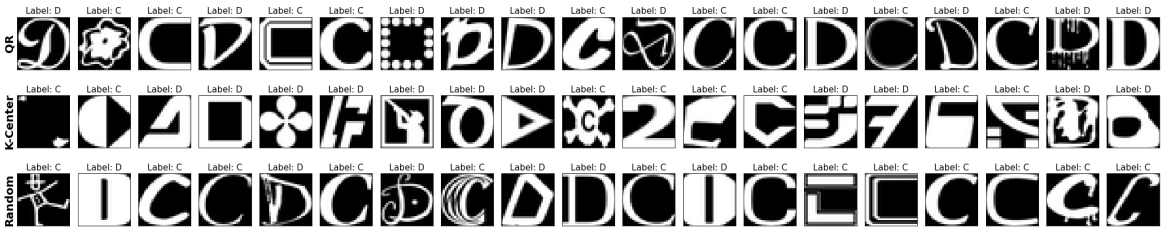


Figure 2: Coreset selection of 20 data points for the second task of Split notMNIST. Evidently, K-Center coreset consists of outliers (second row), while QR chose varied but typical letters (first row), and random being a mix of both (third row).

and admit prior updates. We run 20 epochs of batch sizes 256. Figure 1 presents the average results on tasks over all five model versions. We observe catastrophic forgetting in the case of VCL and VCL+K-Center (see appendix B). Using QRCP on average improves VCL by 0.5. QR Coreset Only shows amazing performance with accuracies similar to VCL methods (based on barely 200 samples from each task).

Coreset Visualisation. We finish with visualizing coreset choices. In figure 2 it is evident that K-Means method tends to choose outliers. This is expected since the algorithm by design selects points furthest from the previous choices. We observe more toned-down selections from QR with a full range of stroke-widths and styles. Observed phenomena would explain the weak perfomance of K-Center on its own and its fine-tuning property on easier datasets when combined with VCL. Further coreset visualisation, including CIFAR-10 and MNIST data, can be found in Appendix B.6.

6 Final Remarks

We have verified the experimental results of [26], observing that the QR coreset selection yields comparable results to K-Center and random methods when combined with VCL. For simpler datasets, VCL+K-Center marginally outperforms by focusing on unique data points, whereas VCL+QRCP dominates in complex datasets by prioritizing typical examples. Interestingly, in models using only the coreset, QR-based choices showcased impressive performance.

Future research should apply VCL to more realistic CL scenarios, addressing the shortcomings of classical benchmarks, particularly moving beyond MNIST and CIFAR datasets, as outlined in [11]. An additional avenue for future work includes exploring CNN architectures that integrate pre-trained lower layers with a few variational ones. It would be also interesting to explore different initializations of the convolutional layers, instead of treating them analogously to the linear ones. Lastly, one critique of QRCP is that by vectorizing the images, it fails to leverage their spatial structure. Naturally then, there is a wide scope for better coreset selection methods for VCL. Given the broad applicability of the framework, it opens the door to incorporating a wide range of episodic memory approaches.

References

[1] E. Anderson, Z. Bai, and J. Dongarra. Generalized QR factorization and its applications. *Linear Algebra and its Applications*, 162-164:243–271, 1992.

[2] Olivier Bachem, Mario Lucic, and Andreas Krause. Coresets for nonparametric estimation - the case of DP-means. In *International Conference on Machine Learning*, 2015.

[3] Mohammadhossein Bateni, Aditya Bhaskara, Silvio Lattanzi, and Vahab Mirrokni. Distributed balanced clustering via mapping coresets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.

[4] J. R. Bischof. A block QR factorization algorithm using restricted pivoting. In *Supercomputing '89:Proceedings of the 1989 ACM/IEEE Conference on Supercomputing*, pages 248–256, 1989.

[5] Yaroslav Bulatov. Notmnist dataset. *Google (Books/OCR), Tech. Rep.[Online]. Available: http://yaroslavvb. blogspot. it/2011/09/notmnist-dataset. html*, 2, 2011.

[6] Peter A. Businger and Gene H. Golub. Linear least squares solutions by householder transformations. *Numerische Mathematik*, 7:269–276, 1965.

[7] Sandipan Chakroborty and Goutam Saha. Feature selection using singular value decomposition and QR factorization with column pivoting for text-independent speaker identification. *Speech Communication*, 52(9):693–709, 2010.

[8] Tony F. Chan and Per Christian Hansen. Some applications of the rank revealing QR factorization. *SIAM Journal on Scientific and Statistical Computing*, 13(3):727–741, 1992.

[9] Zlatko Drmač and Serkan Gugercin. A new selection operator for the discrete empirical interpolation method—improved a priori error bound and extensions. *SIAM Journal on Scientific Computing*, 38(2):A631–A648, 2016.

[10] H. Engler. The behavior of the QR-factorization algorithm with column pivoting. *Applied Mathematics Letters*, 10(6):7–11, 1997.

[11] Sebastian Farquhar and Yarin Gal. Towards robust evaluations of continual learning, 2019. arXiv preprint.

[12] Sebastian Farquhar and Yarin Gal. A unifying bayesian view of continual learning, 2019. arXiv preprint.

[13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, fourth edition, 2013.

[14] Chengcheng Guo, Bo Zhao, and Yanbing Bai. DeepCore: A comprehensive library for coreset selection in deep learning, 2022.

[15] Jonathan Huggins, Trevor Campbell, and Tamara Broderick. Coresets for scalable bayesian logistic regression. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.

[16] I. T. Jolliffe. *Principal Component Analysis*. Springer-Verlag, New York, 2 edition, 2002.

[17] Heechul Jung, Jeongwoo Ju, Minju Jung, and Junmo Kim. Less-forgetting learning in deep neural networks, 2016.

[18] KrishnaTeja Killamsetty, Xujiang Zhao, Feng Chen, and Rishabh K. Iyer. RETRIEVE: coreset selection for efficient and robust semi-supervised learning. *CoRR*, abs/2106.07760, 2021.

[19] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017.

[20] Jeremias Knoblauch, Hisham Husain, and Tom Diethe. Optimal continual learning has perfect memory and is NP-hard, 2020.

[21] Alex Krizhevsky. Learning multiple layers of features from tiny images. *University of Toronto*, 05 2009.

[22] Zhizhong Li and Derek Hoiem. Learning without forgetting, 2017.

[23] Noel Loo, Siddharth Swaroop, and Richard E. Turner. Generalized variational continual learning, 2020.

[24] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. volume 24 of *Psychology of Learning and Motivation*, pages 109–165. Academic Press, 1989.

[25] Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. Coresets for data-efficient training of machine learning models, 2020.

[26] Cuong V. Nguyen, Yingzhen Li, Thang D. Bui, and Richard E. Turner. Variational continual learning. In *International Conference on Learning Representations*, 2018.

[27] Benjamin Peherstorfer, Zlatko Drmač, and Serkan Gugercin. Stability of discrete empirical interpolation and gappy proper orthogonal decomposition with randomized and deterministic sampling points. *SIAM Journal on Scientific Computing*, 42(5):A2837–A2864, 2020.

[28] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. iCaRL: Incremental classifier and representation learning, 2017.

[29] Mark B. Ring. *Continual Learning in Reinforcement Environments*. Ph.d. dissertation, University of Texas at Austin, 1994. Available from ProQuest Dissertations and Theses database. (UMI No. 304133181).

[30] Anthony Robins. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2):123–146, 1995.

[31] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks, 2022.

[32] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach, 2018.

[33] Samarth Sinha, Han Zhang, Anirudh Goyal, Yoshua Bengio, Hugo Larochelle, and Augustus Odena. Small-GAN: Speeding up GAN training using core-sets, 2019.

[34] Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning, 2019.

[35] Kai Wei, Rishabh K. Iyer, and Jeff A. Bilmes. Submodularity in data subset selection and active learning. In *International Conference on Machine Learning*, 2015.

[36] Jianwei Xiao, Ming Gu, and Julien Langou. Fast parallel randomized QR with column pivoting algorithms for reliable low-rank matrix approximations. In *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*. IEEE, December 2017.

[37] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3987–3995. PMLR, 06–11 Aug 2017.

Appendix

A Theory

A.1 Approximating posterior with variational inference

We consider a simple setting with only the last layer W to optimize, to explain the concept. Recall the *Kullback-Leiber* (KL) similarity measure between distributions

$$\mathrm{KL}(q,p) = \int q(x) \log \frac{q(x)}{p(x)}$$

where p often corresponds to the true distribution and q an attempt to model it. We want to approximate the posterior $p(W|X,Y)$ of model weights W using some distribution $q_\theta(W)$ parameterized by θ. We will asses the quality by minimizing the KL divergence between the two.

Approximate inference To use it as a optimization objective in practice, we need to find a way of *evaluating* the above. Note that

$$\begin{aligned}
&\mathrm{KL}(q_\theta(W), p(W|X,L)) \\
&= \int q_\theta(W) \log \frac{q_\theta(W)}{p(W|X,Y)} dW \\
&= \int q_\theta(W) \log \frac{q_\theta(W) p(Y|X)}{p(W) p(Y|X,W)} dW \\
&= \int q_\theta(W) \Big[\log \frac{q_\theta(W)}{p(W)} + \log p(Y|X) - \log p(Y|X,W) \Big] dW \\
&= \mathrm{KL}(q_\theta(W), p(W)) + \log p(Y|X) - \int q_\theta(W) \log p(Y|X,W) dW.
\end{aligned}$$

Where $p(Y|X,W)$ is the *likelihood*, $p(W)$ is the *prior* and $p(Y|X)$ is the *model evidence*. Then, note that we can extract the terms that don't depend on θ. We arrive at the following objective

$$\max_\theta [\int q_\theta(W) \log p(Y|X,W) dW - \mathrm{KL}(q_\theta(W), p(W))] \tag{4}$$

with first term denoting how well we explain the data (if possible, q should put all mass at MLE). Second term evaluates how close q_θ is to the prior (if possible, q should be prior itself).

Moreover, KL ≥ 0, so we have $\log(p(Y|X)) \geq (4)$ which provides an upper bound and is referred to as *evidence lower bound* (ELBO). Note that the KL of the approximation of posterior is the 'slack' in our bound - the tighter the bound the smaller the slack and approximation improves.

Stochastic Approximate Inference Assuming a simple Gaussian likelihood, the term in 4 can be explicitly calculated. The likelihood, in practice is much more complex than Gaussian. Instead of this assumption and explicit calculation, we turn to *stochastic integration*.

$$\mathbb{E}_{q_\theta(W)}[\log p(Y|X,W)] = \int q_\theta(W) \log p(Y|X,W) dW$$

is then approximated using *Monte Carlo Integration* (MC) as

$$\hat{\mathbb{E}}_{q_\theta(W)}[\log p(Y|X,W)] = \sum_{W \sim q_\theta(W)} \log p(Y|X,W) dW.$$

In Deep Neural Networks, upon arrival of data points $\{x_i\}_1^N$ and predictions $\{y_i\}_1^N$, we define the loss function of the prior:

$$\mathcal{L}(q_\theta(W)) = \mathrm{KL}(q_\theta(W), p(W)) + \sum_{n=1}^{N} \mathbb{E}_{q_\theta(W)}[\log p(y_i|x_i,W)] \tag{5}$$

with the first term easily evaluated in closed form because of additive property of KL. The derivative of second term can be approximated using MC and the *local reparametrization trick*.

A.2 Posterior Updates

We have:

$$\begin{aligned}
p(\theta|D_{1:T}) &= \frac{p(\theta)p(D_{1:T}|\theta)}{p(D_{1:T})} \qquad \text{(Bayes rule)} \\
&\propto p(\theta)p(D_{1:T}|\theta) \\
&= p(\theta)p(D_T|\theta)p(D_{1:T-1}|\theta) \qquad \text{(by independence)} \\
&= p(\theta)p(D_T|\theta)\frac{p(D_{1:T-1})p(\theta|D_{1:T-1})}{p(\theta)} \qquad \text{(Bayes rule)} \\
&\propto p(D_T|\theta)p(\theta|D_{1:T-1})
\end{aligned}$$

A.3 Deriving the Evidence Lower Bound

Assume that we want to approximate the posterior $p(\theta|\mathcal{D})$ by minimizing the KL-divergence with a simpler distribution q. We have the following derivations:

$$\begin{aligned}
\mathrm{KL}(q(\theta)||p(\theta|\mathcal{D})) &= \int q(\theta)\log\frac{q(\theta)}{p(\theta|\mathcal{D})}d\theta \\
&= \int q(\theta)\log\frac{q(\theta)p(\mathcal{D})}{p(\theta)p(\mathcal{D}|\theta)}d\theta \\
&= \int q(\theta)\Big[\log\frac{q(\theta)}{p(\theta)} + \log p(\mathcal{D}) - \log p(\mathcal{D}|\theta)\Big]d\theta \\
&= \mathrm{KL}(q||p) + \log p(\mathcal{D}) - \mathbb{E}_{q(\theta)}[\log p(\mathcal{D}|\theta)].
\end{aligned}$$

We can omit $\log p(\mathcal{D})$ and arrive at the objective of maximizing

$$\mathbb{E}_{q(\theta)}[\log p(\mathcal{D}|\theta)] - \mathrm{KL}(q||p)$$

with respect to parameters defining q.

B Experiments

Code for preparing the datasets was inspired by this GitHub repository `https://github.com/Piyush-555/VCL-in-PyTorch/tree/main`.

B.1 Architectural details

Coreset Only Implementation of Coreset Only algorithm was not clearly specified in the original paper [26]. Here, we admit the VFE loss function without prior updates and initial prior $q_0 = \mathcal{N}(0, I)$. With lack of prior updates, we set $q_t(\theta) = q_0(\theta)$ and the term $\mathrm{KL}(q_t(\theta)||q_{t-1}(\theta))$ from $\mathcal{L}_{VCL}$ is always zero.

CNN Architecture for the Split CIFAR-10 task

1. 3x3 Bayesian convolutional layer with 16 filters, padding of 1, ReLu activation
2. 2x2 Max Pooling with stride 2
3. 3x3 Bayesian convolutional layer with 16 filters, padding of 1, ReLu activation
4. 2x2 Max Pooling with stride 2
5. Flattening
6. Fully connected Bayesian layer with 576 units and ReLU activation
7. Task-specific head layers with 150 hidden units

B.2 Coreset Selection Methods

QR with Column Pivoting algorithm I have implemented the QR-CP algorithm in the torch library. Specifically for this application the first pivot is selected at random since otherwise, the method would select the max-norm image. Recall that the algorithm decomposes a matrix A into a product of three matrices $AP = QR$, where Q is an orthogonal matrix, R is upper triangular and P is a permutation matrix that rearranges columns of A. Algorithm proceeds iteratively, by choosing a max-norm pivot; the corresponding column is then swapped with the first one and used to eliminate the elements below the diagonal using Householder orthogonal projection. The process repeats until the matrix is transformed into an upper-triangular one. At the end of the algorithm, P indicates the order of selected columns, capturing the most significant features of the original matrix in a hierarchical manner.

K-Center greedy algorithm K-Center is implemented in a vectorized form for efficiency. The algorithm runs as follows: begins by arbitrarily choosing an initial data point; iterates through the dataset, calculating for each point the distance to its nearest selected center; the point with the maximum distance to its nearest center is then chosen as the next center; continues iteratively until a total of K centers are selected.

This method ensures that the selected centers are optimally spread across the dataset, capturing the diversity and structure of the data, thereby forming a representative coreset.

B.3 Spit MNIST Experiment

On Figure 3 we can see methods performing very similarly in the Split MNIST task. In the methods using coresets QR-CP seems to be best choice, and K-Center being least successful. VCL presents best accuracy with K-Center method that helps it re-train on outliers.

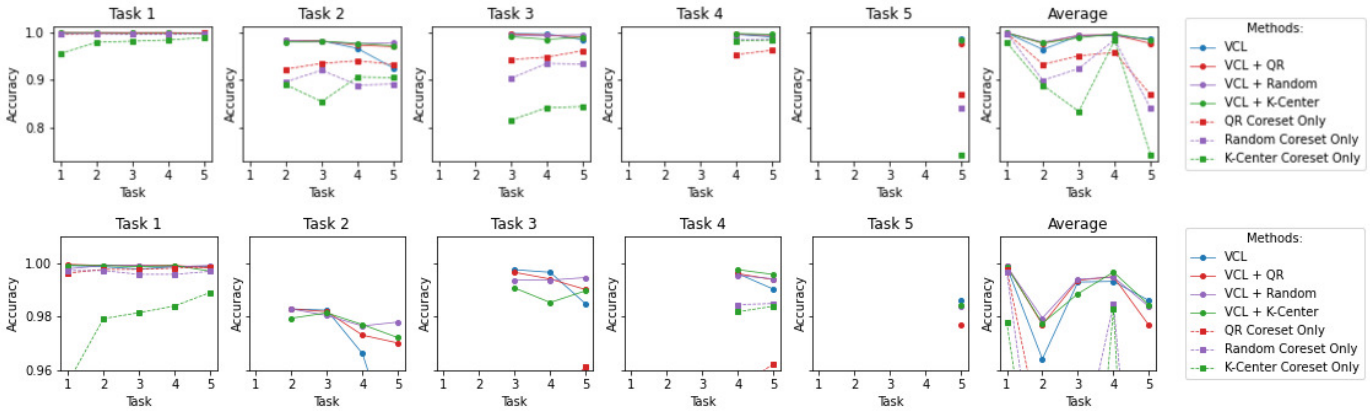


Figure 3: Split MNIST experiment with VCL based-methods and corset variations. Plots show accuracy on the five tasks as they changed with model refinement. Lower plot zooms-in to show more detail.

B.4 Split notMNIST Experiment

Figure 4 presents the experimental results described in section 5. We can see a very similar performance of the coreset methods when combined with VCL. Plot from Task 1 showcases the catastrophic forgetting of pure VCL model, that drops to 0.6 (which in case of a binary classification is equivalent to random guessing).
When used on their own, QR-CP and random coreset methods present a surprisingly good performance of 0.9 accuracy. Recall that these were trained on merely 40 examples per task.

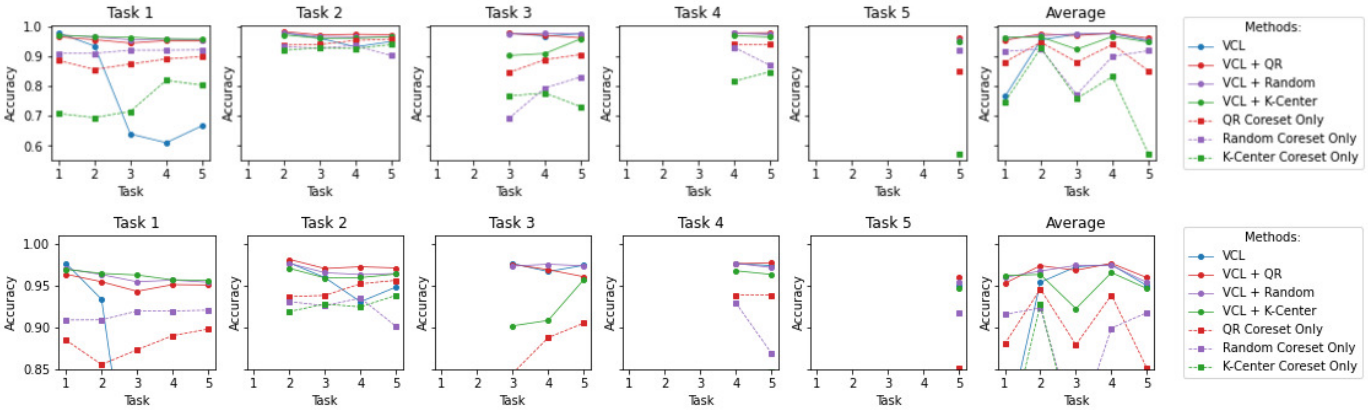


Figure 4: Split notMNIST experiment with VCL based-methods and corset variations. Plots show accuracy on the five tasks as they changed with model refinement. Lower plot zooms-in to show more detail.

B.5 Split CIFAR-10 Experiment

Figure 5 presents breakdown of accuracies between tasks as model develops. There is an apparent forgetting pattern with the VCL and VCL+K-Center models.The random and QR versions of VCL produce models that didn't change accuracy with time. There doesn't seem to be an all-best method since they showcase very similar performances. Perhaps extending the training to more iterations would aid in such distinction.

When the methods used only the coresets for training, the overall performance is unsatisfactory. However, QR-CP shows a visible advantage over the two alternatives.

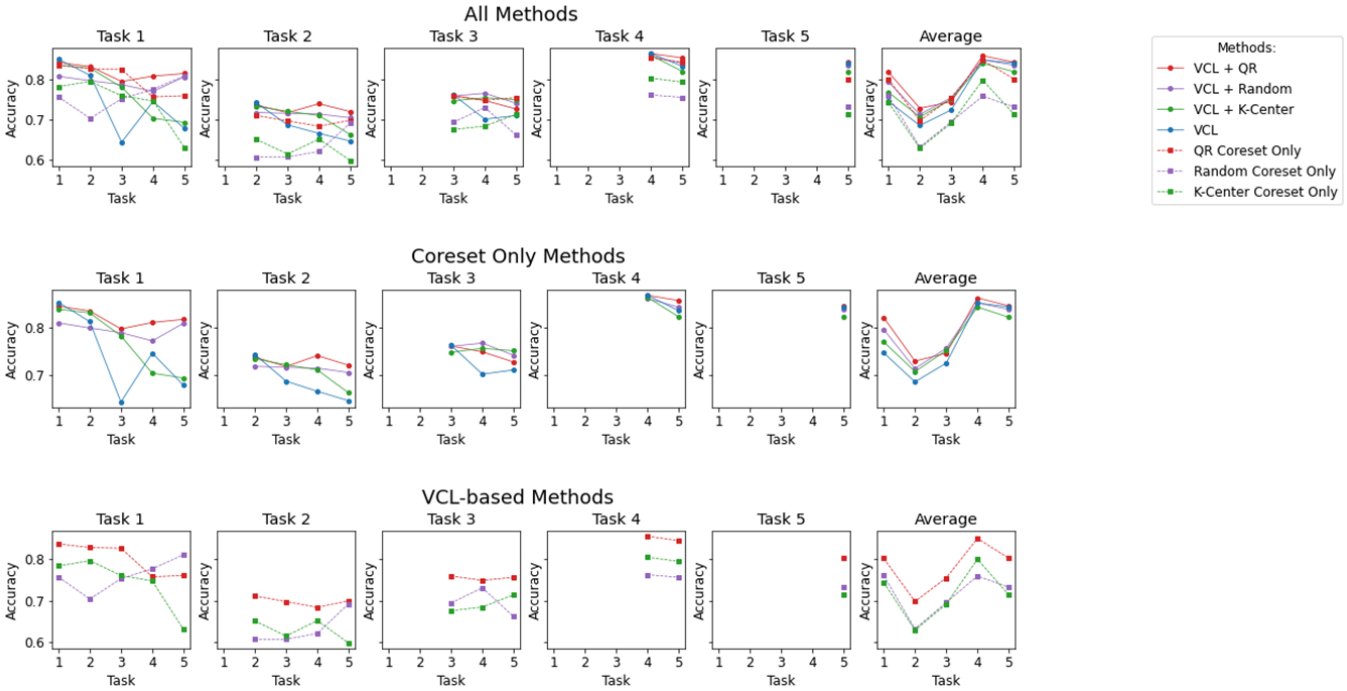


Figure 5: Split CIFAR-10 experiment with VCL based-methods and corset variations. Plots show accuracy on the five tasks as they changed with model refinement. Lower plots show the two method classes separately for visibility.

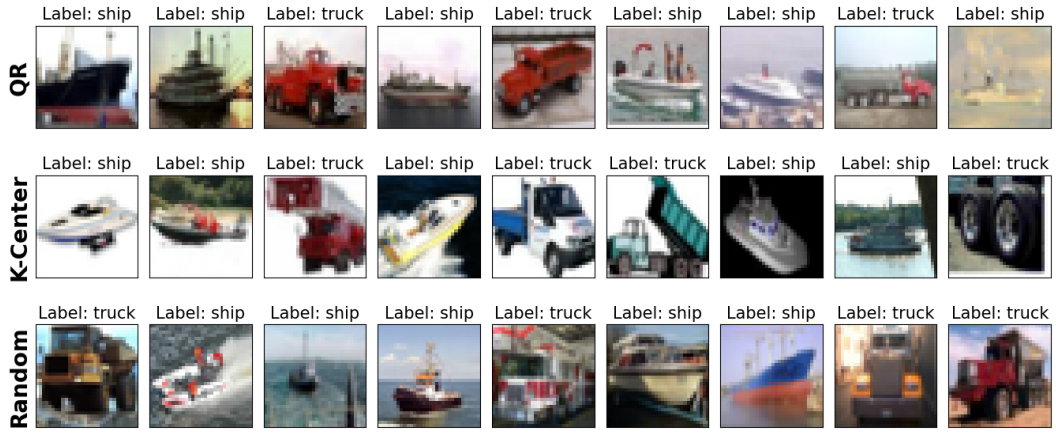


Figure 6: Coreset method selections from the car/airplane task in the CIFAR-10 dataset. Note the evident outliers in the K-Center choices (middle row), including darkest and lightest images and unrepresentative one - the truck wheel.

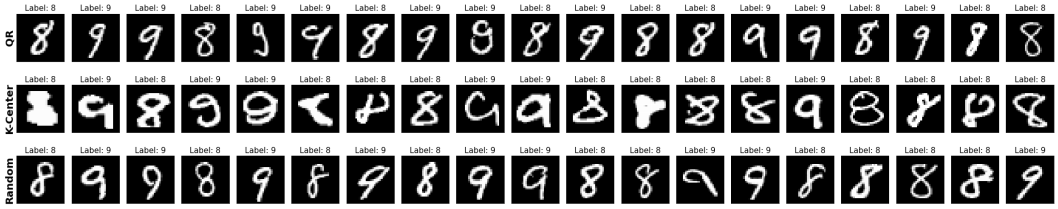


Figure 7: Coreset method selections from the 8/9 task in the MNIST dataset. Note the evident outliers in the K-Center choices (middle row), and lack of outliers in QR ones (first row)

B.6 Coreset Visualisation

We visualize the coreset method choices to inspect our intuition about their properties. See figure 6 for Split CIFAR-10 task 5 (airplane/car) selections. See figure 7 for MNIST task 5 (numbers 8/9) selections. By inspection, we notice tendency of K-Center method to choose outliers, while QR concentrated on more typical data points. Note also that random seems to give a variety of both.

We note that the first coreset choice has been omitted in the visualisations since it is a random element in implementations of each selection methods.